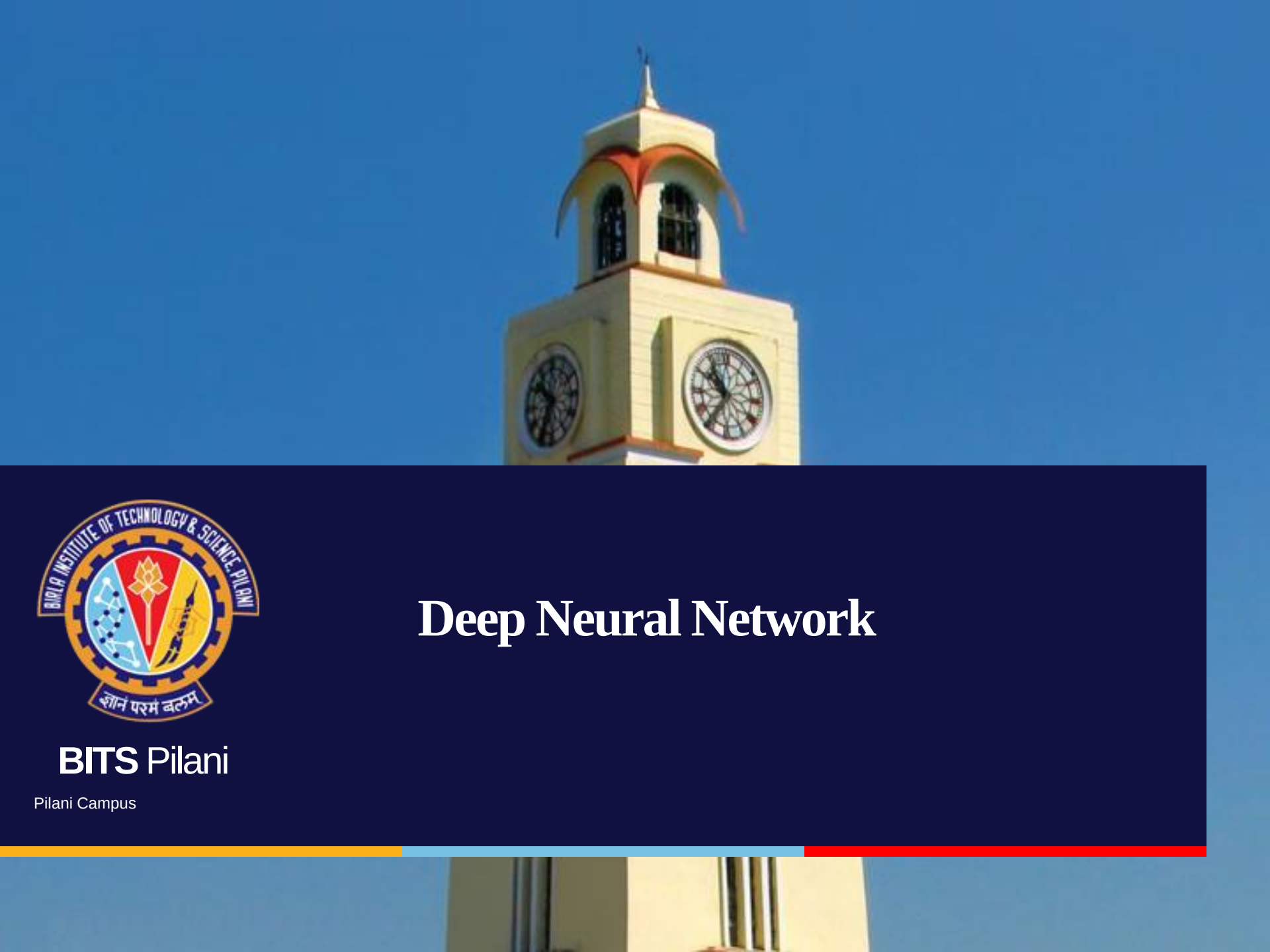


Deep Neural Network

BITS Pilani

Pilani Campus

Deep Neural Network

Disclaimer and Acknowledgement

- The content for these slides has been obtained from books and various other source on the Internet
- I here by acknowledge all the contributors for their material and inputs.
- I have provided source information wherever necessary
- I have added and modified the content to suit the requirements of the course

Session Agenda

innovate achieve lead

- Activation functions
- Weight initialization

Nonlinearity in MLP

- A nonlinear Multi-Layer Perceptron (MLP) is a type of neural network with multiple layers and nonlinear activation functions.
- In contrast to a linear model, which can only model linear relationships in data, a nonlinear MLP can capture complex, nonlinear patterns and relationships.
- These activation functions allow the network to model complex relationships in the data that wouldn't be possible with linear transformations alone.

Activation function

- Activation function of a neuron defines the output of that neuron given an input or set of inputs.
- Activation functions decide whether a neuron should be activated or not by calculating the weighted sum and adding bias with it.
- They are differentiable operators to transform input signals to outputs, while most of them add non-linearity.
- Artificial neural networks are designed as universal function approximators, they must have the ability to calculate and learn any nonlinear function.

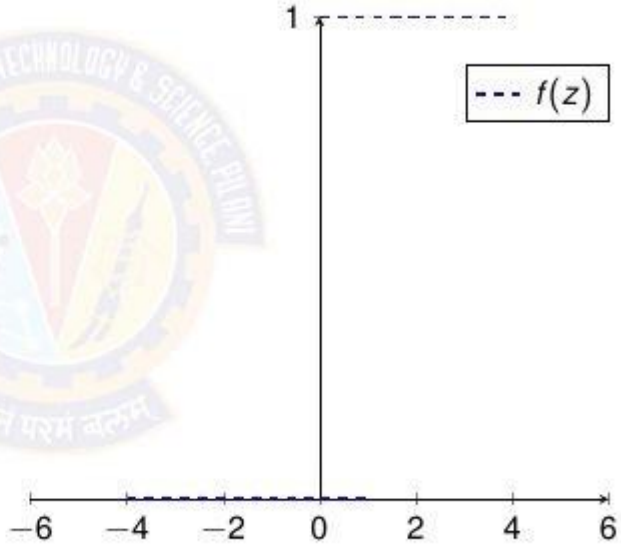
1. Step Function

$$\text{Function: } f(z) = \begin{cases} 0 & \text{for } z \leq 0 \\ 1 & \text{for } z > 0 \end{cases}$$

$$\text{Derivative: } f'(z) = 0$$

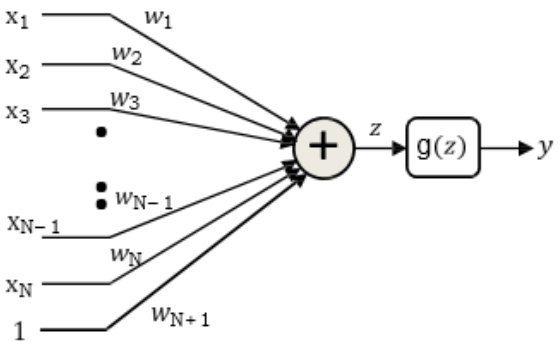
$$\text{Range} : (-\infty, +\infty)$$

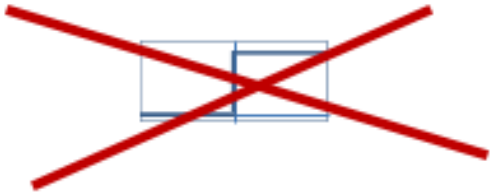
- Linear function
- Derivative is 0. So neuron cannot learn.

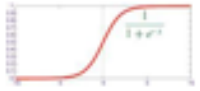


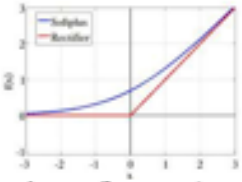
Differentiable activation functions

- The perceptron is a flat function with zero derivative everywhere, except at 0 where it is non-differentiable
  - You can vary the weights a *lot* without changing the error
  - **There is no indication of which direction to change the weights to reduce error**
- Lets make the neuron differentiable, with non-zero derivatives over much of the input space
  - Small changes in weight can result in non-negligible changes in output
  - This enables us to **estimate the parameters using gradient descent techniques..**









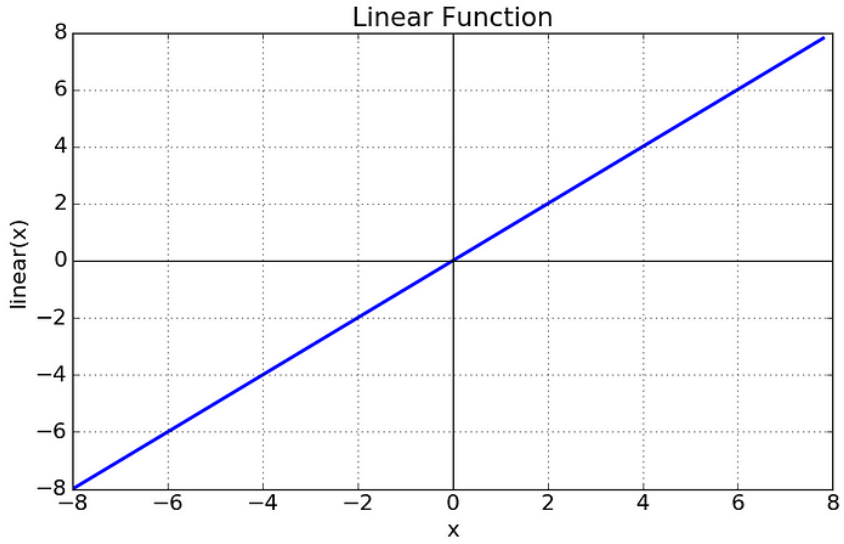
Activation functions g(z)

2. Linear activation function

- Also known as "no activation," or "identity function" (multiplied x1.0)
- Activation is proportional to the input.
-

$$f(x) = x$$

$$f'(x) = 1$$



3. Sigmoid function

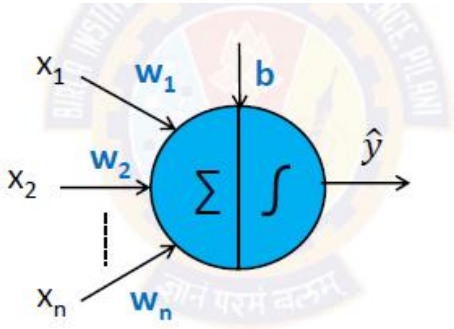
- Often referred to as the logistic sigmoid
- **S-shaped** curve transforms the input into an output in the range of (0, 1)
- When the neuron's activation are 0 or 1, sigmoid neurons saturate Gradients at these regions are almost zero (almost no signal will flow)
- Here's an explanation of how the sigmoid activation function works:

Input-Output Transformation: The sigmoid function takes an input value (x) and transforms it into an output value ($f(x)$) using the following formula:

$f(x) = \frac{1}{1+e^{-x}}$

- e is the base of the natural logarithm (approximately 2.71828).
- For very large positive values of x, e^{-x} becomes very small, making $f(x)$ close to 1.
- For very large negative values of x, e^{-x} becomes very large, making $f(x)$ close to 0.

S-Shaped Curve: The sigmoid function has an S-shaped curve, which means that it responds smoothly to changes in the input. This characteristic allows it to capture gradual, non-linear relationships between input and output.

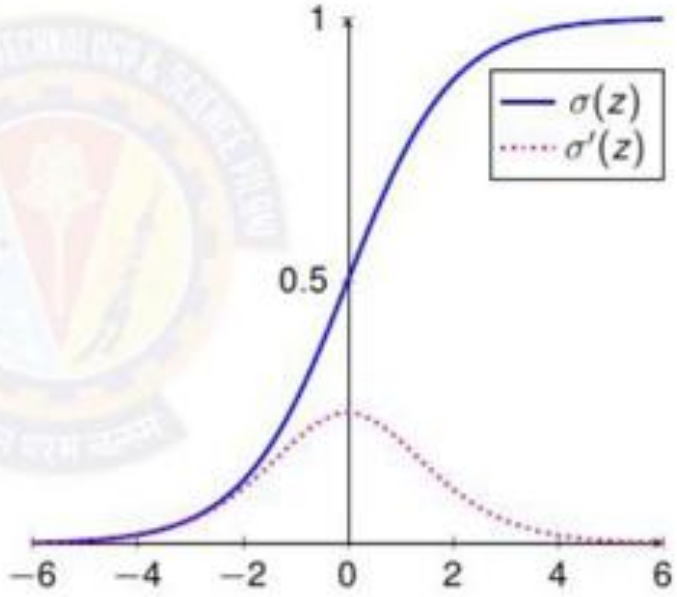


3. Sigmoid function

Function: $\sigma(z) = \dfrac{1}{1 + e^{-z}}$

Derivative: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

Range : $(0, 1)$



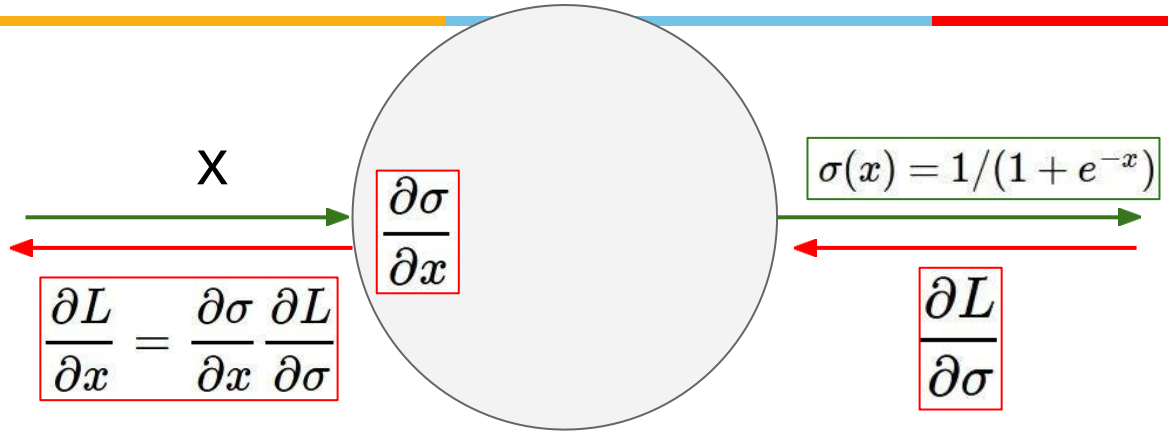
- Non-Linear function
- Small changes in z will be large in $f(z)$. This means it is a good classifier.
- Leads to vanishing gradient. So the learning is minimal.

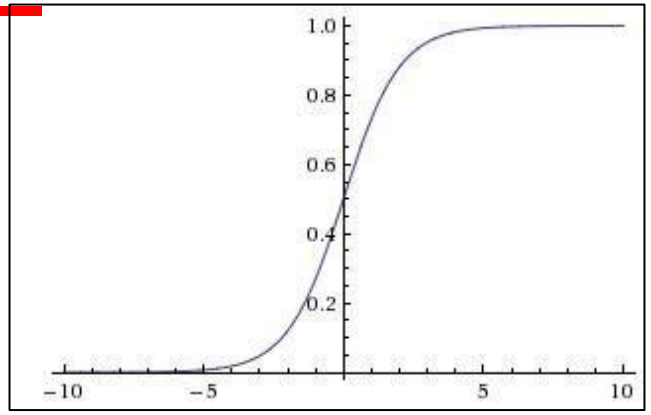
Squashing function as: it squashes any input in the range (-inf, inf) to some value in the range (0, 1)

Drawbacks of Sigmoid

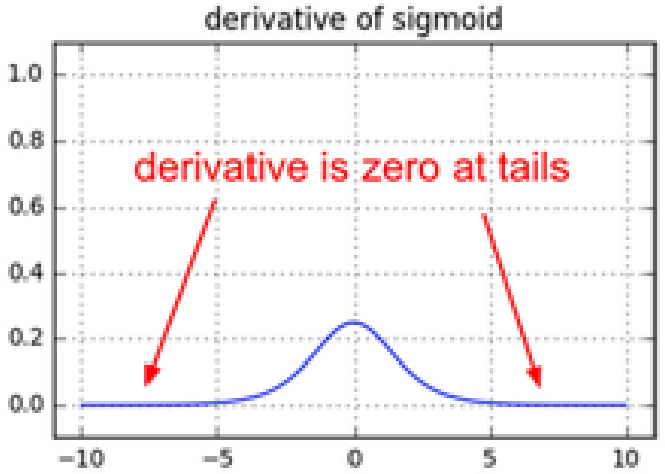
- High computational complexity
  - The sigmoid function contains the exponential operation. Thus, the computation is inefficient
- Neuron Saturation problem
- Vanishing gradient problem
- Nonzero-centered outputs

Neuron Saturation problem





What happens when x = -10?
What happens when x = 0?
What happens when x = 10?



Saturated neurons "kill" the gradients

When gradients are zero, during gradient descent, weights will not update

Vanishing gradient problem

- Derivative of the sigmoid function is in the range of 0, 0.25
- Absolute values of the initialized weights are generally less than 1,
- Gradient would approach 0, thus causing the **vanishing gradient problem.**

$\sigma`(z^{[2]})$

$$\frac{\partial \mathcal{L}}{\partial z^{[2]}} = \frac{\partial \mathcal{L}}{\partial a^{[2]}} \times \frac{\partial a^{[2]}}{\partial z^{[2]}} = a^{[2]} - y$$

$$\frac{\partial \mathcal{L}}{\partial w^{[1]}} = \frac{\partial \mathcal{L}}{\partial a^{[2]}} \times \frac{\partial a^{[2]}}{\partial z^{[2]}} \times \frac{\partial z^{[2]}}{\partial a^{[1]}} \times \frac{\partial a^{[1]}}{\partial z^{[1]}} \times \frac{\partial z^{[1]}}{\partial w^{[1]}} = \frac{\partial \mathcal{L}}{\partial z^{[1]}} \; x^T$$

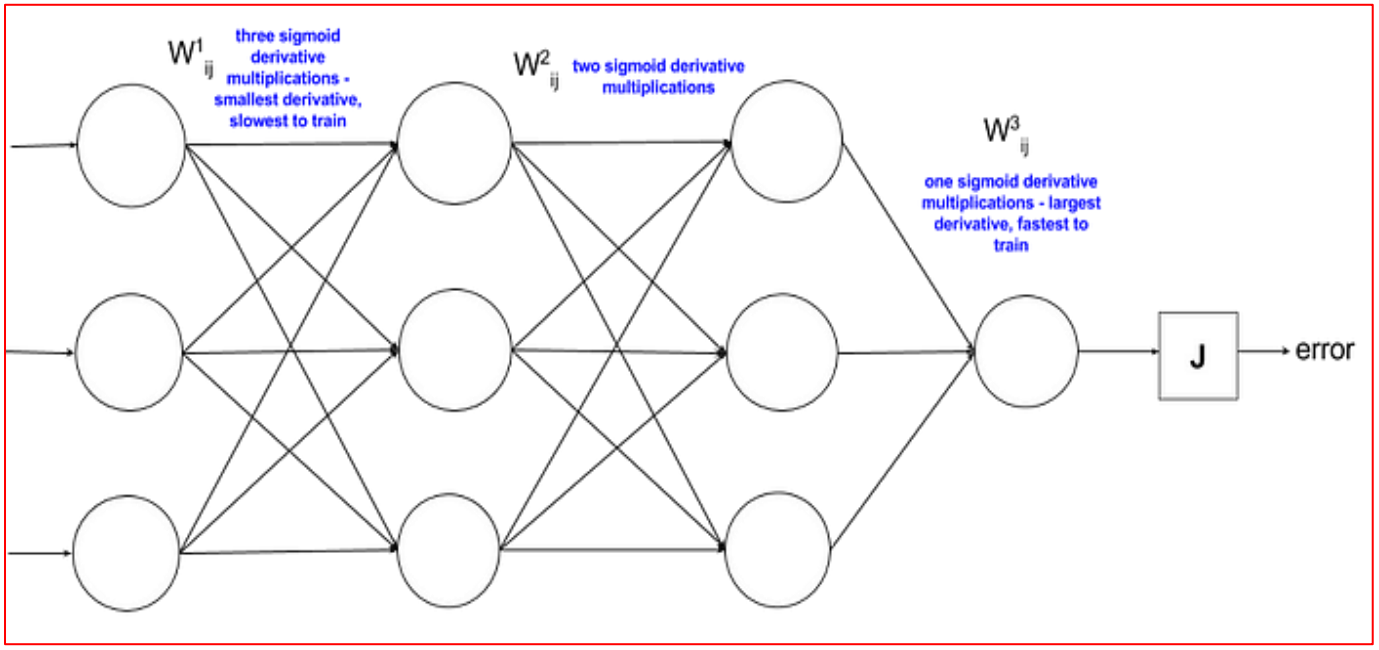
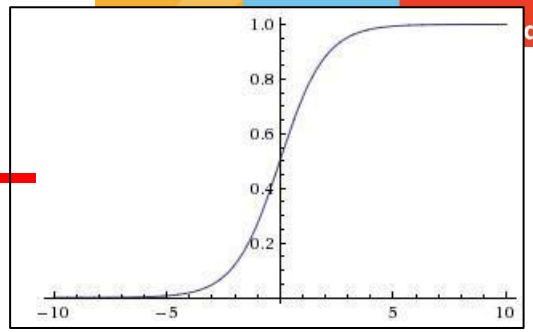
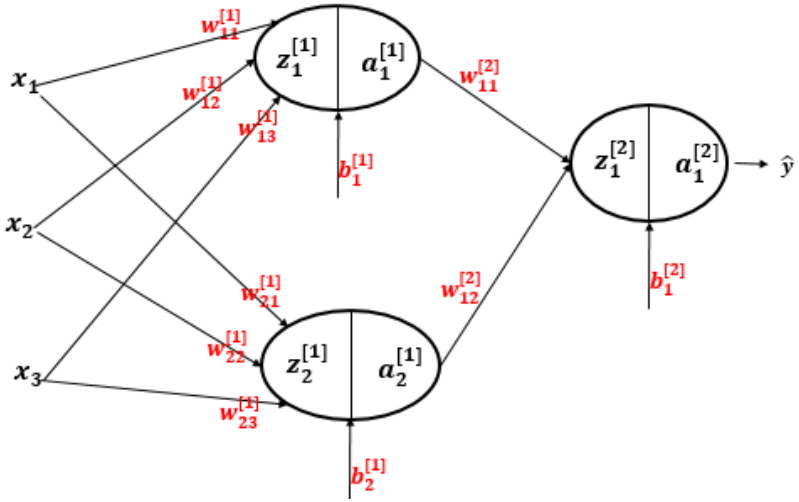


Image credit: https://ayearofai.com/rohan-4-the-vanishing-gradient-problem-ec68f76ffb9b

Nonzero-centered outputs



- Consider what happens when the input to a neuron is always positive:



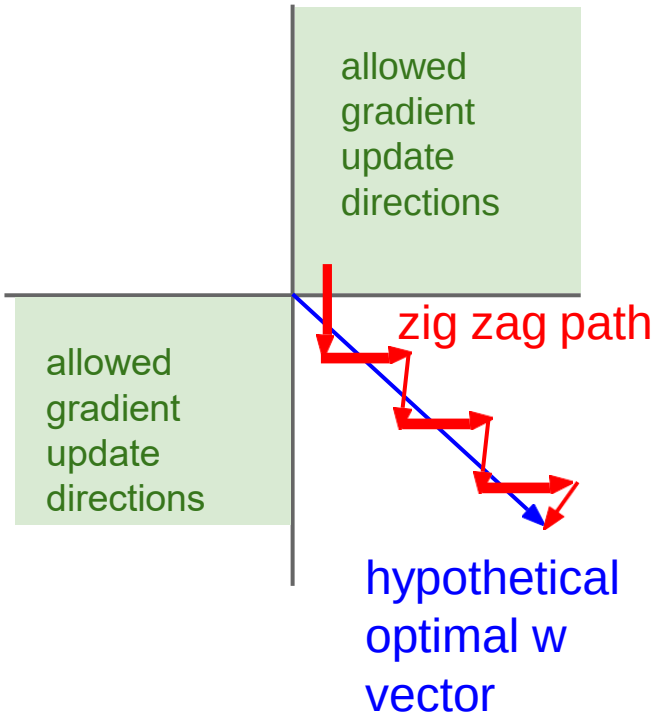
$$dw_{11}^{[2]} = \frac{\partial \mathcal{L}}{\partial W_{11}^{[2]}} = dz_1^{[2]} \cdot a_1^{[1]}$$

$$dw_{12}^{[2]} = \frac{\partial \mathcal{L}}{\partial W_{12}^{[2]}} = dz_1^{[2]} \cdot a_2^{[1]}$$

Always positive why?

What can we say about the gradients on **w**?

Nonzero-centered outputs



- What can we say about the gradients on **w**? Always all positive or all negative :
- (this is also why you want zero-mean data!)
- Affects the network convergence

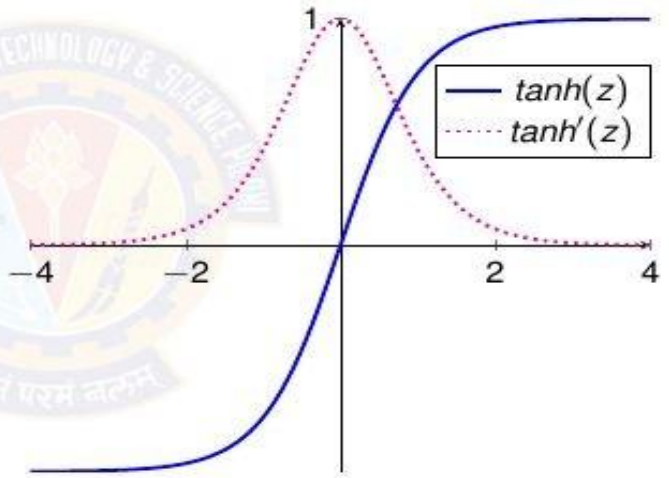
4.Hyperbolic Tangent (tanh) Function

- Like sigmoid it has S shaped curve and models non linear relationship in data
- Squashes numbers to range [-1,1]
- Zero centered (nice) – faster convergence, hence uses in hidden layers
- still kills gradients when saturated
- Still computationally expensive

$$\text{Function: } tanh(z) = \frac{(e^z - e^{-z})}{(e^z + e^{-z})}$$

$$\text{Derivative: } tanh'(z) = (1 - tanh^2(z))$$

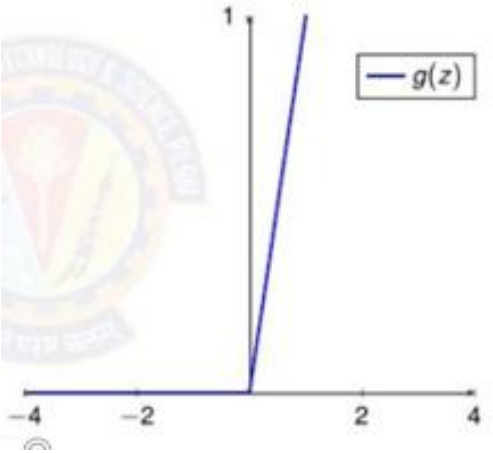
$$\text{Range} : (-1, 1)$$



- This looks very similar to sigmoid. In fact, it is a scaled sigmoid function!

$$tanh(x) = 2\, sigmoid(2x) - 1$$

5. Relu (rectified linear unit) Function

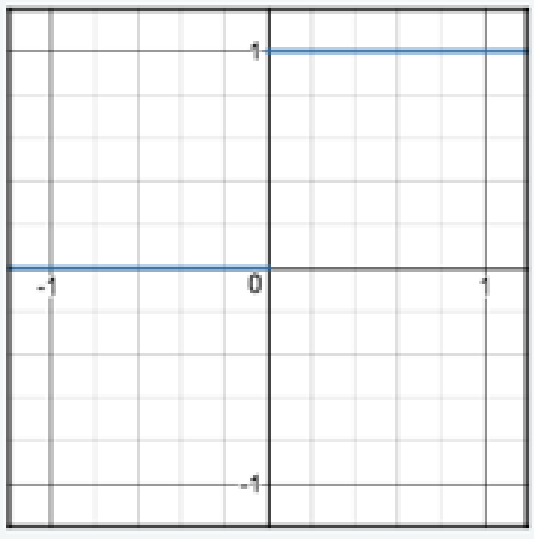


Rectified Linear Unit

Function: $g(z) = \max(0, z)$

Derivative: $g'(z) = \begin{cases} 0 & \text{for } z \leq 0 \\ 1 & \text{for } z > 0 \end{cases}$

Range : $[0, \infty]$



- Because the rectified function is linear for half of the input domain and nonlinear for the other half, it is referred to as a piecewise linear function or a hinge function
- Kills negative values; thus, stops further propagation of negative values, hence known as rectifier

5. Relu

Advantages:

- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)
- Sparse activation (in a randomly initialized network, only about 50% of hidden units are activated i.e have a non-zero output)

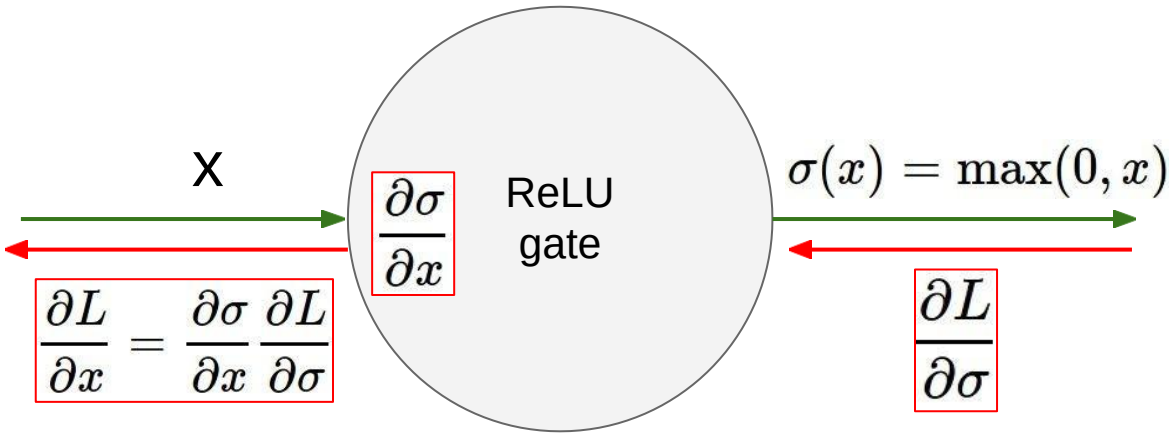
Disadvantages:

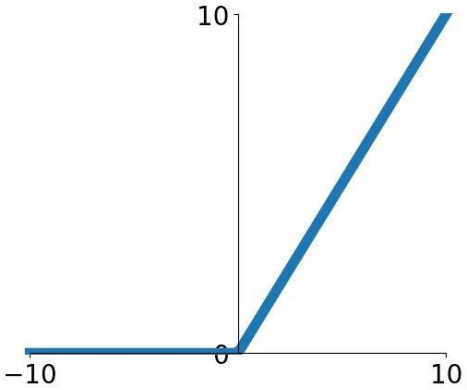
- Non-differentiable at zero (not a major problem as neural network training algorithms do not usually arrive at a local minimum of the cost function)
- Not zero-centered: Unbounded – (use Batch mormalization)
- Dying ReLU problem – (use leaky Relu)

Dying Relu

hint: what is the gradient when x < 0?

This is a form of the vanishing gradient problem



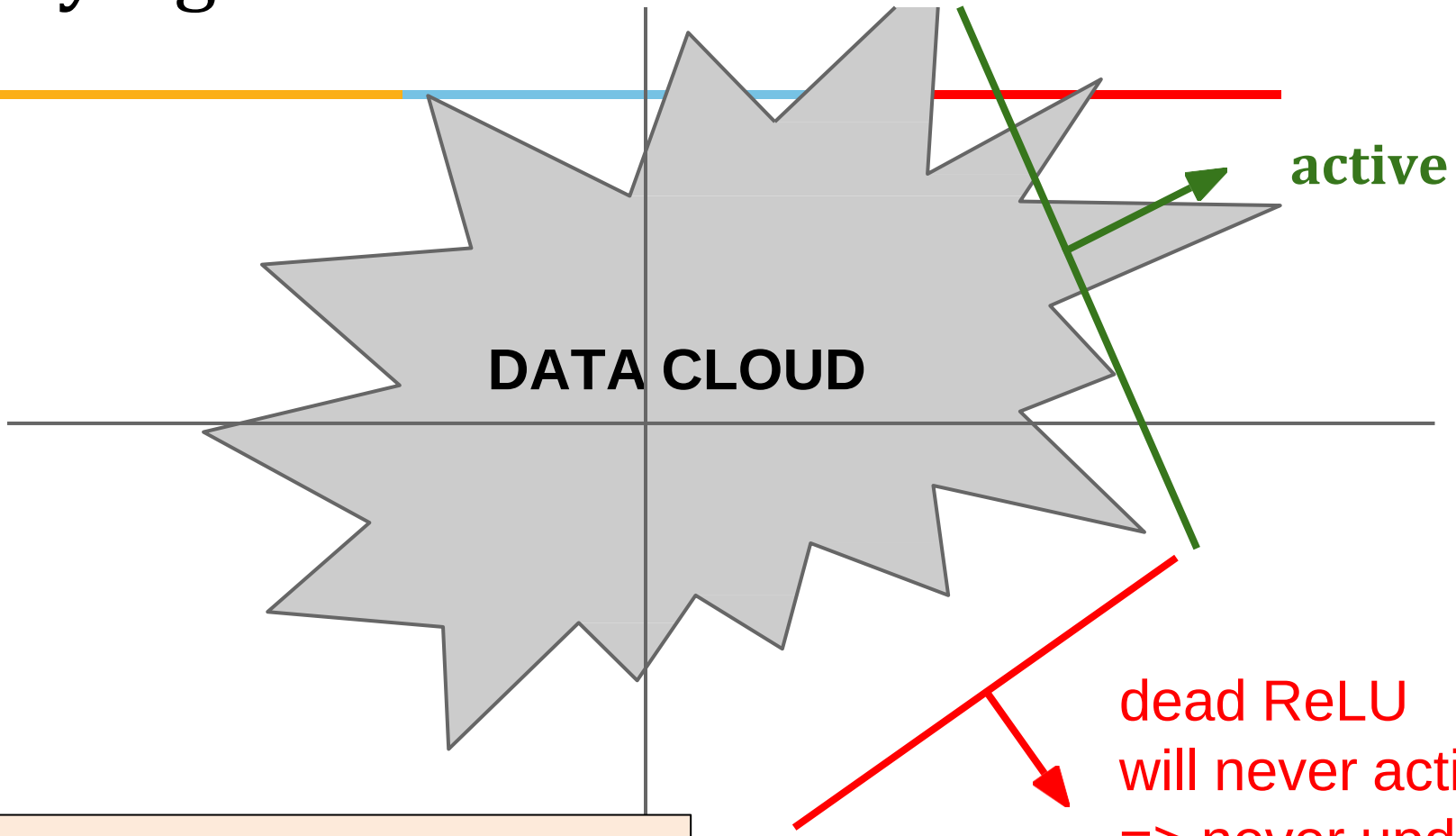


What happens when x = -10?

What happens when x = 0?

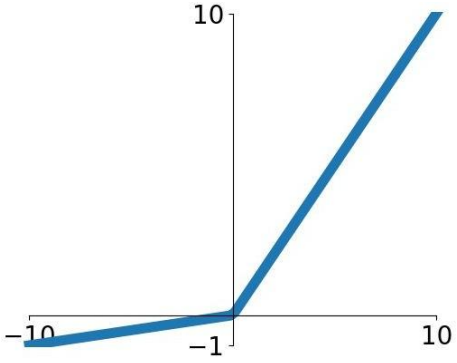
What happens when x = 10?

Dying Relu



=> people like to initialize ReLU neurons with slightly positive biases (e.g. 0.01)

Leaky ReLU



$$f(x) = \max(0.01x, x)$$

- Does not saturate
- Computationally efficient
- Converges much faster than sigmoid/tanh in practice! (e.g. 6x)
- **will not "die".**

Parametric Rectifier (PReLU)

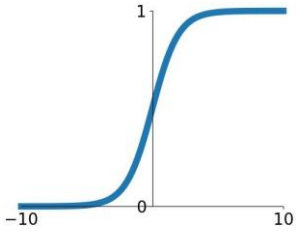
$$f(x) = \max(\alpha x, x)$$

backprop into \alpha (parameter)

Activation Functions: Summary

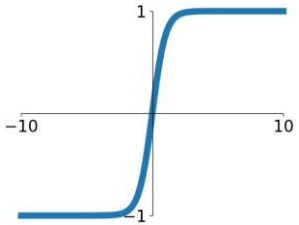
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



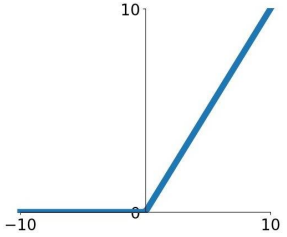
tanh

$$\tanh(x)$$



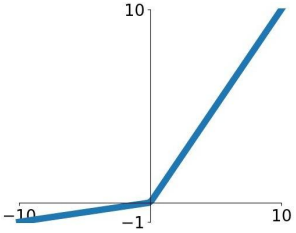
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

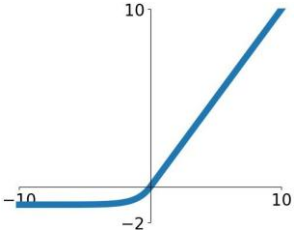


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

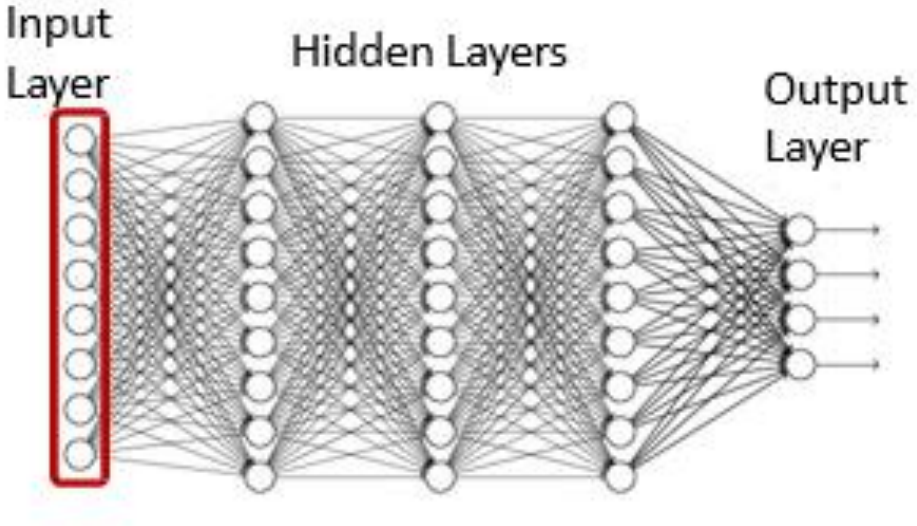


Activation Functions: Summary

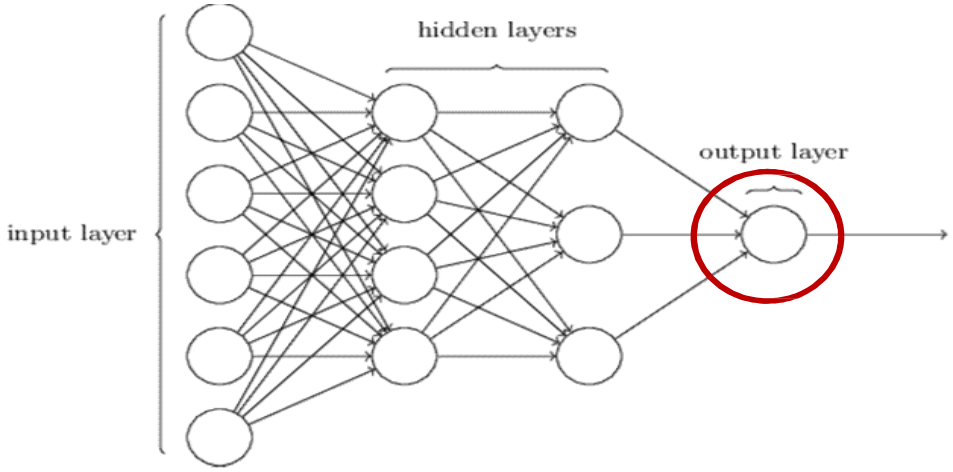
Activation Function	Sigmoid	Tanh	ReLU
Linearity	Non Linear	Non Linear	Non Linear
Activation Function	$\sigma(x) = \frac{1}{1+e^{-z}}$	$tanh(z) = \frac{(e^z - e^{-z})}{(e^z + e^{-z})}$	$g(x) = \max(0, z)$
Derivative	$\sigma(z)(1 - \sigma(z))$	$(1 - tanh^2(z))$	$1\ if\ z > 0\ else\ 0$
Symmetric Function	No	Yes	No
Range	$[0, 1]$	$[-1, 1]$	$[0, \infty]$
Vanishing Gradient	Yes	Yes	No
Exploding Gradient	No	No	Yes
Zero Centered	No	Yes	No

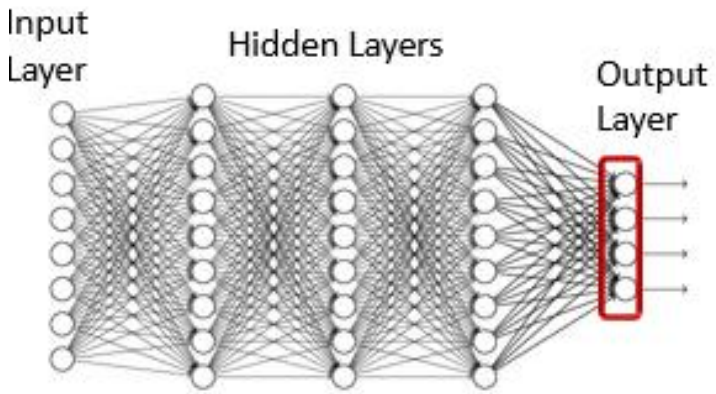
Representing the input

- Vectors of numbers
  - (or may even be just a scalar, if input layer is of size 1)
  - E.g. vector of pixel values
  - E.g. vector of speech features
  - E.g. real-valued vector representing text
  - Other real valued vectors



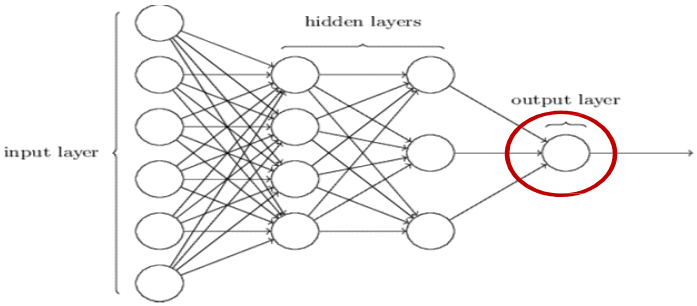
Representing the output

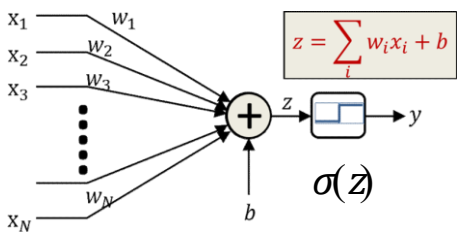


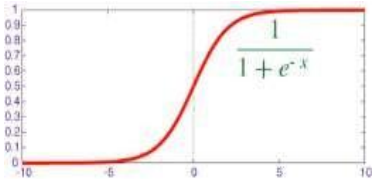


- If the desired *output* is real-valued, no special tricks are necessary
  - Scalar Output : single output neuron
    - d = scalar (real value)
  - Vector Output : as many output neurons as the dimension of the desired output
    - $d = [d_1\ d_2\ ..\ d_L]$ (vector of real values)

Representing the output : binary class







- If the desired output is binary (is this a cat or not), use a simple 1/0 representation of the desired output
  - 1 = Yes it's a cat
  - 0 = No it's not a cat
- Output activation: Typically a sigmoid
  - Viewed as the *probability* $P(Y=1|X)$ *o*f class value 1
    - Indicating the fact that for actual data, in general a feature value X may occur for both classes, but with different probabilities
    - Is differentiable

Multi-class output: One-hot representations

- Consider a network that must distinguish if an input is a cat, a dog, a camel, a hat, or a flower
- We can represent this set as the following vector:

  [cat dog camel hat flower]^T
- For inputs of each of the five classes the desired output is:

  cat: [1 0 0 0 0]^T

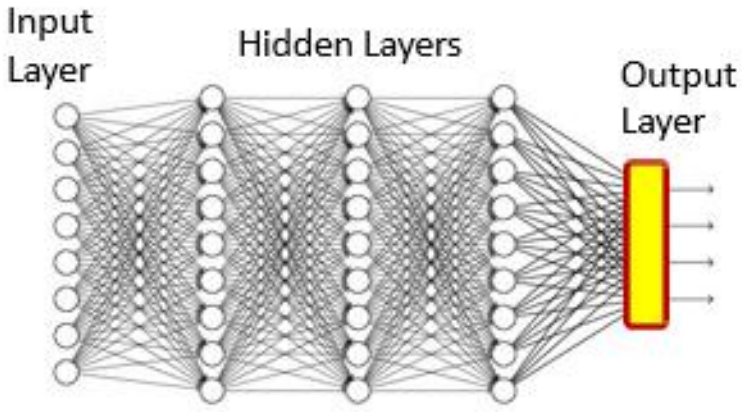
  dog: [0 1 0 0 0]^T

  camel: [0 0 1 0 0]^T

  hat: [0 0 0 1 0]^T

  flower: [0 0 0 0 1]^T
- For an input of any class, we will have a five-dimensional vector output with four zeros and a single 1 at the position of that class
- This is a *one hot vector*

Multi-class networks

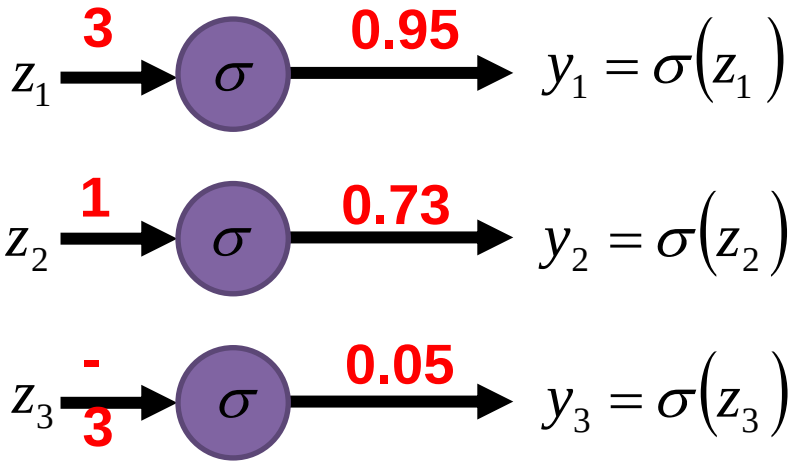


$$z_i^{[l]} = \sum_j w_{ij}^l \; a_j^{l-1} + b_i^l$$

$$a_i^{[l]} = \frac{e^{z_i}}{\sum_j e^{z_j}} \rightarrow P(class = i|X)$$

- For a multi-class classifier with N classes, the one-hot representation will have N binary target outputs
- The neural network's output too must ideally be binary (N-1) zeros and a single 1 in the right place)
- More realistically, it will be a probability vector
  - N probability values that sum to 1.
- Softmax vector activation is often used at the output of multi-class classifier nets

A sigmoid Layer



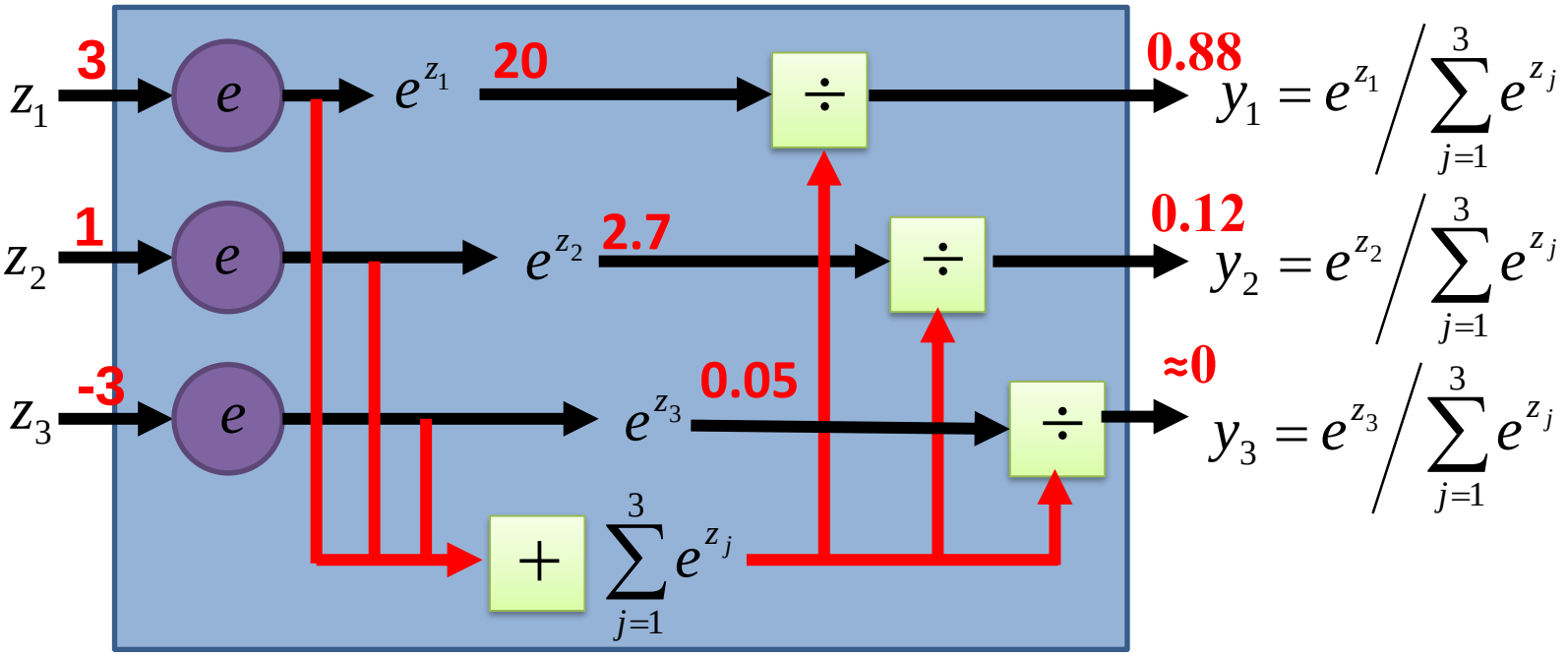
Softmax Layer

The values z inputted to the softmax layer are referred to as ***logits***

Probability:

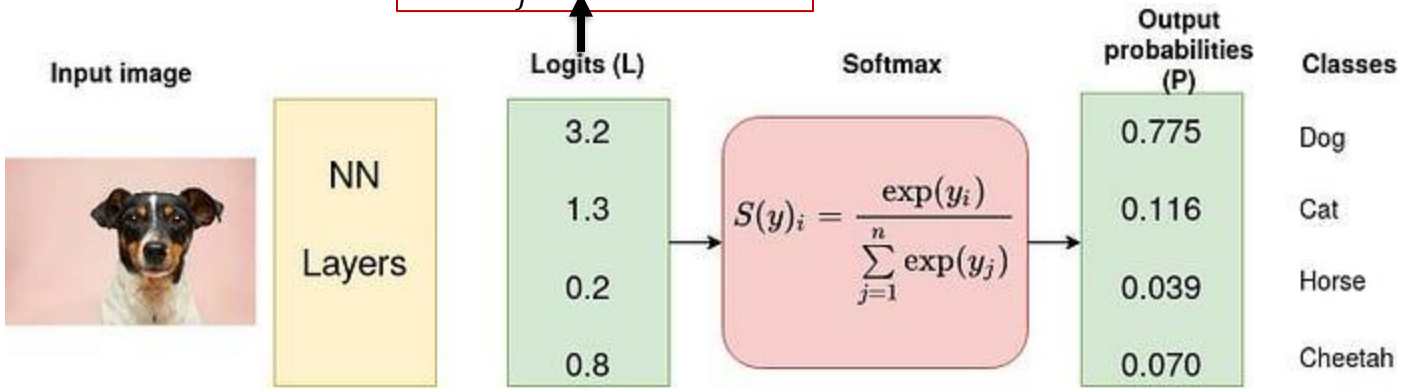
- $0 < y_i < 1$
- $\sum_i y_i = 1$

A Softmax Layer



Multiclass classification example

$$z_i^{[l]} = \sum_j w_{ij}^l \; a_j^{l-1} + b_i^l$$



$$e^{3.2} = 24.5325$$
$$e^{1.3} = 3.6693$$
$$e^{0.2} = 1.2214$$
$$e^{0.8} = 2.2255$$

$$S(3.2) = \frac{e^{3.2}}{e^{3.2} + e^{1.3} + e^{0.2} + e^{0.8}}$$

$$= 0.775$$

Example credit: https://towardsdatascience.com/softmax-activation-function-how-it-actually-works-d292d335bd78

Multi-class cross entropy loss

- Binary cross entropy is calculated on top of sigmoid outputs, whereas Categorical cross-entropy is calculated over softmax activation outputs.

$$L(\hat{y}, y) = -\sum_{k}^{K} y^{(k)} \log \hat{y}^{(k)}$$

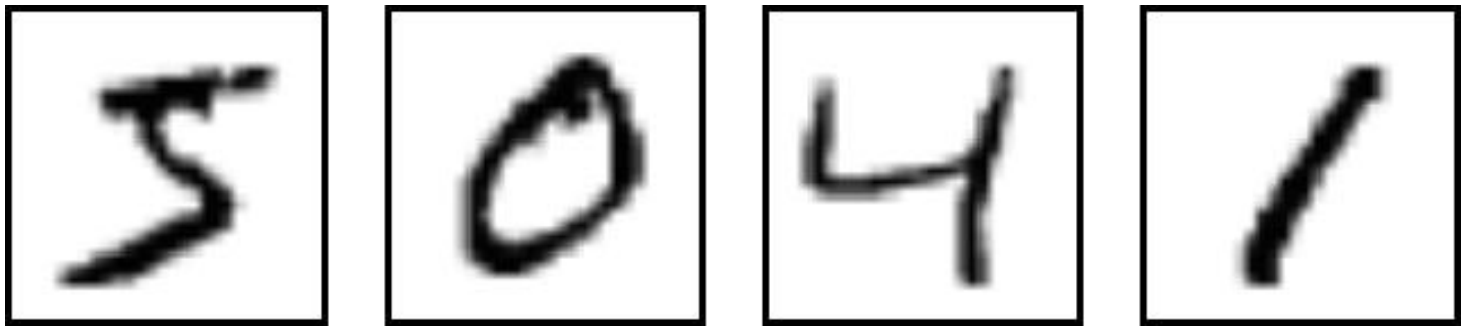
- Here, $y^{(k)}$ is 0 or 1, indicating whether class label k is the correct classification for predicting $\hat{y}^{(k)}$

- With softmax activation

$$L(\hat{y}, y) = -\sum_{k}^{K} y^{(k)} \log \frac{e^{\hat{y}^{(k)}}}{\sum_{j=1}^{K} \hat{e}^{\hat{y}^{(j)}}}$$

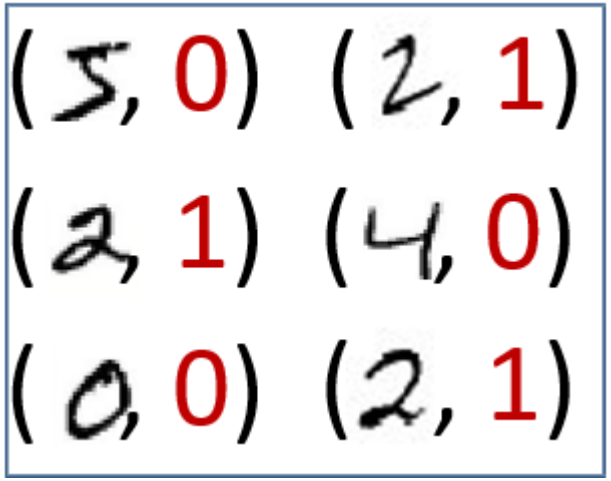
Typical Problem Statement

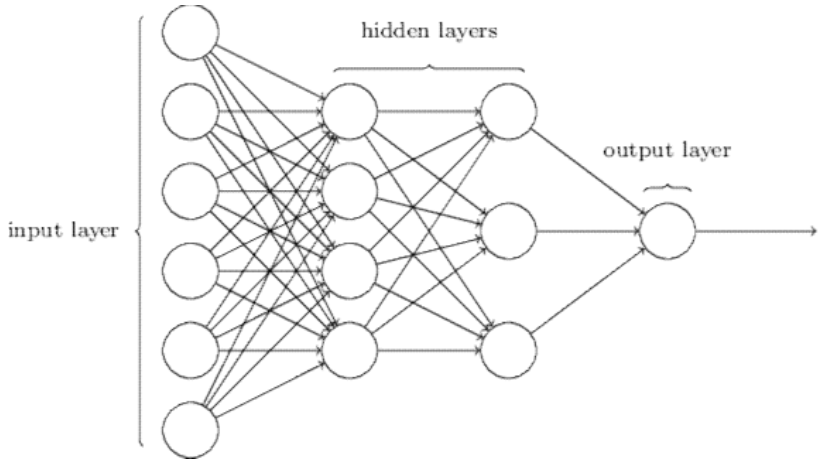
- We are given a number of "training" data instances
- E.g. images of digits, along with information about which digit the image represents
- Tasks:
  - Binary recognition: Is this a "2" or not
  - Multi-class recognition: Which digit is this? Is this a digit in the first place?



Typical Problem statement : **binary classification**

Training data

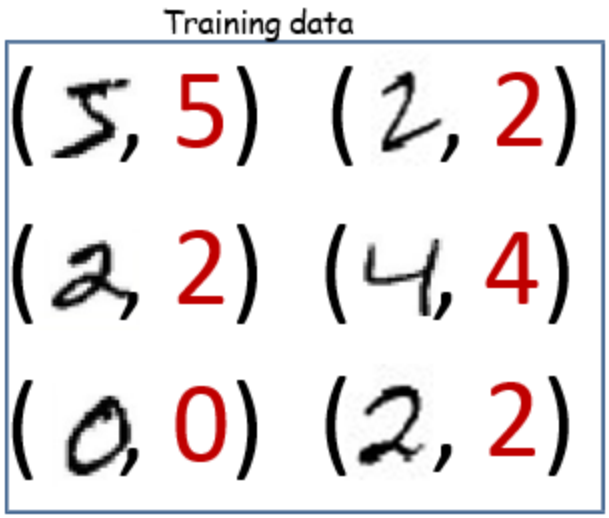


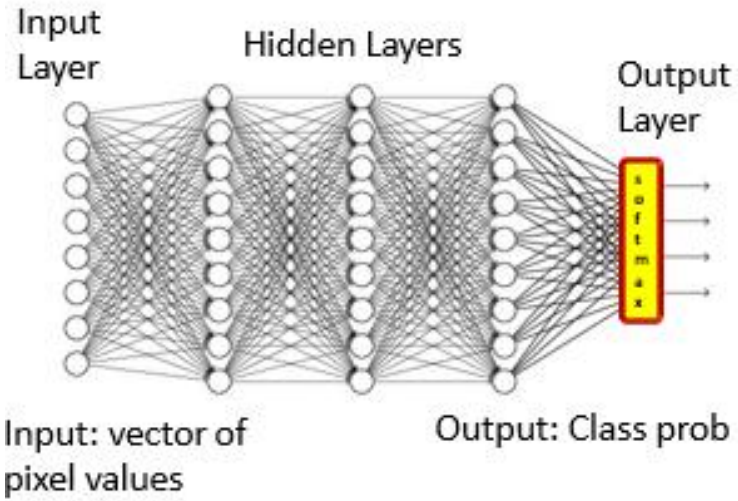


Input: vector of pixel values

Output: sigmoid

Typical Problem statement: **Multiclass classification**



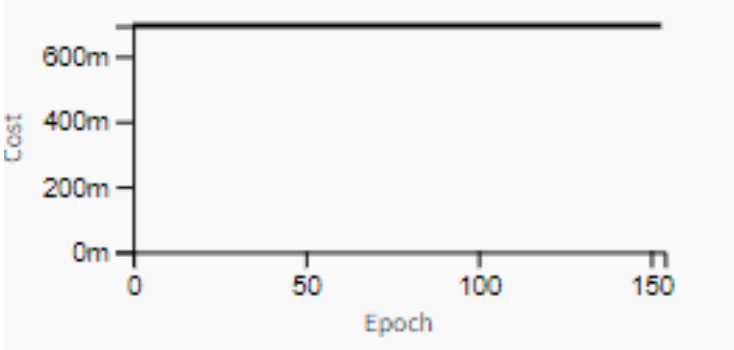


Initializing neural networks

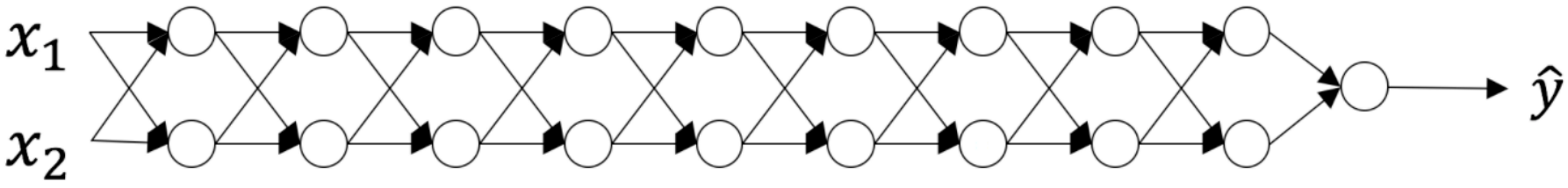
- The choice of initialization is crucial for maintaining numerical stability.
- The choices of initialization can be tied up in interesting ways with the choice of the nonlinear activation function.
- Which function we choose and how we initialize parameters can determine how quickly our optimization algorithm converges.
- Poor choices can cause to encounter exploding or vanishing gradientswhile training.

Initializing neural networks

- Zero initialization for weights
  - All the neurons learn the same features during training
  - hidden units will have identical influence on the cost, which will lead to identical gradients.



Initializing neural networks



Consider linear activation for the above NN, weights are all matrices of size (2,2)

$$\hat{y} = a^{[L]} = W^{[L]}W^{[L-1]}W^{[L-2]}\dots W^{[3]}W^{[2]}W^{[1]}x$$

A too-large initialization

$$\begin{bmatrix} 1.5 & 0 \\ 0 & 1.5 \end{bmatrix}$$

- Values of $a^{[l]}$ increase exponentially with l
- Gradients explodes due to large activations leads to the **exploding gradient problem**
- Cost oscillates around its min.

A too-small initialization

$$\begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$$

- values of $a[l]$ decrease exponentially with l
- Gradients vanishes due to small activations leads to the **vanishing gradient problem**
- Fails to converge

Xavier Initialization

- Samples weights from a Gaussian distribution with zero mean and variance

$$Var[W^i] = \frac{2}{n_i + n_{i+1}}$$

- n_i = size of i^{th} layer

- Now-standard and practically beneficial

https://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf?hc_location=ufi

References

- Ref TB- Dive into Deep Learning chapter 5

Thank You All !